OpenZeppelin | security

TJAR Diff Audit



August 3, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Security Model and Trust Assumptions 6

Additional Considerations 7

Low Severity 8

L-01 Reentrancy During Asset Sweep Breaks Nonce Execution Guarantee 8

Notes & Additional Information 9

N-01 Incomplete Docstrings 9

N-02 Missing Docstrings 10

N-03 Unused Named Return Variables 10

N-04 Resource Recipient May Be Misconfigured As The Releaser 10

N-05 Typographical Error 11

Conclusion 12

Appendix 13

Issue Classification 13

Summary

Type	DeFi
Timeline	From 2026-07-29 To 2026-07-31
Languages	Solidity

Total Issues	6 (6 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	1 (1 resolved)
Notes & Additional Information	5 (5 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin conducted a diff audit of the Uniswap/tjar repository covering the changes between base snapshot af81c35 and target snapshot 4a90fe8.

In scope were the following modified or added files:

```
src
    ├── base
    │   ├── Nonce.sol
    │   └── ResourceManager.sol
    ├── interfaces
    │   ├── base
    │   │   ├── INonce.sol
    │   │   ├── IOwned.sol
    │   │   └── IResourceManager.sol
    │   ├── external
    │   │   ├── IL2StandardBridge.sol
    │   │   └── IOptimismMintableERC20.sol
    │   ├── IFactory.sol
    │   ├── IFeeController.sol
    │   ├── IReleaser.sol
    │   └── ITokenJar.sol
    ├── releasers
    │   ├── ExchangeReleaser.sol
    │   └── OptimismBridgedResourceReleaser.sol
    ├── Factory.sol
    ├── FeeController.sol
    └── TokenJar.sol
```

System Overview

TJAR is a fee collection and redemption system for protocol revenue. Each partner deployment consists of a `TokenJar`, which passively accumulates native currency and ERC-20 tokens from arbitrary sources, and a Releaser, which is the only contract permitted to move assets out of that jar. Redemption is permissionless: any party may pay a fixed amount of a configured token and receive the jar's accumulated balances in exchange, which removes the need for keepers, price oracles, or manual intervention.

`ExchangeReleaser` implements that exchange. A redeemer supplies a nonce, the list of assets to sweep, a recipient, and an upper bound on the protocol fee. The releaser debits the redeemer a fixed `threshold` amount of the configured resource token, forwards it to the resource recipient, charges an optional protocol fee on top of it, and then instructs the jar to transfer the full balance of each listed asset to the recipient. The release configuration is mutable only as a unit, through a single `setConfig` call, and every configuration change advances a sequential nonce so that a release priced against superseded terms reverts instead of executing against terms its caller never saw.

`OptimismBridgedResourceReleaser` extends that flow for OP Stack L2 deployments whose collected revenue belongs on L1. It pins the resource recipient to itself and, after each release, initiates a withdrawal of its full resource balance through the L2 Standard Bridge to a configurable L1 recipient. The resource token is frozen at construction, because the two stage flow depends on that token remaining withdrawable through the standard bridge.

Two singletons support every deployment. `FeeController` holds a chain-wide protocol fee, expressed in basis points of a release threshold and bounded by a compile-time maximum of ten percent, which releasers read at the time of each release. `Factory` deploys jar and releaser pairs through CREATE2 using salts scoped to the calling address, wires the pair together, appoints the configuration roles, and transfers ownership of both contracts to the partner, retaining no authority over them afterwards.

Security Model and Trust Assumptions

Jar Owner: holds the sole authority to set or rotate the jar's releaser, which is the only address permitted to move assets out of the jar. A rotation takes effect immediately, and a release in flight against the previous releaser reverts before any asset moves. Partners relying on a particular set of release terms, including burn configurations that direct revenue to a burn address, must trust the jar owner not to rotate to a releaser with different terms. A burn commitment is consequently a property of the jar ownership together with the releaser configuration, rather than of the releaser alone. The jar owner can also appoint an externally owned account as the releaser, which would permit that account to withdraw the jar's balances without any payment.

Releaser Owner: sets the complete release configuration, consisting of the resource token, the resource recipient, the threshold, and the maximum number of assets per release. The configuration is written atomically and always advances the release nonce. The owner additionally appoints the configuration setter and, in the bridged variant, sets the L1 resource recipient. Because the threshold is denominated in the resource token, the owner determines the price at which the jar can be emptied, and lowering it relative to the value of the resource permits the jar to be redeemed below the value of its contents.

Configuration Setter: an optional delegated role carrying the same configuration authority as the releaser owner. It can redirect the resource recipient, or replace the resource and threshold together, so it can direct the proceeds of every subsequent release to an address of its choosing or reprice a release arbitrarily within the validation constraints. It must therefore be trusted to the same degree as the releaser owner.

Protocol Fee Owner: owns the chain-wide `FeeController` and sets both the fee, in basis points and bounded by the compile-time maximum, and the fee recipient. Releasers read the fee at the time of each release and it is not covered by the release nonce, so redeemers bound it through an explicit maximum fee parameter. Because the fee transfer sits on the path of every release, a fee recipient that cannot receive a given resource token blocks releases for every deployment using that token until the fee is set to zero or the recipient is changed, during which time revenue accumulates in the jars.

Redeemers: permissionless participants rather than an on-chain role. A redeemer selects which assets to sweep, the recipient of those assets, and the upper bound on the protocol fee,

and relies on the sequential nonce to establish that the terms simulated at a given nonce still hold at execution. Because the asset list is supplied by the caller and the jar performs a transfer for each listed asset, redeemers are responsible for the tokens they choose to include.

Deployers: any party may deploy jar and releaser pairs through the `Factory`, which holds no privileges over the contracts it deploys. CREATE2 salts are scoped to the calling address, so a given predicted address is reachable only by the same deployer using the same salt.

Additional Considerations

- The bridged releaser hard-codes the OP Stack L2 Standard Bridge predeploy address, which is identical across OP Stack chains. Its withdrawals settle on L1 only after the bridge's challenge period, and the bridge binds each initiated withdrawal to the L1 recipient captured at initiation, so a later change of that recipient does not redirect withdrawals already in flight.
- Address parity across chains depends on initcode being byte identical. `FeeController` takes its initial owner as a constructor argument that must be passed identically on every chain, with per-chain governance ownership established afterwards through a transfer, and `Factory` takes the fee controller address as its own constructor argument. Jar address parity additionally requires the same deployer address on each chain, so a deployer whose address does not commit to its controller could allow an unintended party to claim a predicted address on another chain.
- Directing funds to a predicted but undeployed jar address assumes that the factory is deployed at its canonical address on that chain and that the same deployer identity performs the deployment. Neither assumption is enforceable on-chain.
- The behavior of the resource token, including upgradeability, pausing, denylisting, and transfer fees, sits outside the guarantee provided by the release nonce and is trusted for the life of a configuration. A resource that deducts a fee on transfer delivers less than the configured threshold, because a release debits the redeemer a fixed amount rather than verifying the amount received.
- Ownership across the system is transferred in a single step, so a transfer to an incorrect address cannot be recovered.

Low Severity

L-01 Reentrancy During Asset Sweep Breaks Nonce Execution Guarantee

The `Nonce` contract documents that the sequential nonce allows a searcher who simulates a release at a given nonce to execute it without another transaction landing first. However, `handleNonce` validates and increments the nonce before the function body runs, forming a pre-condition with no matching post-condition, and `release` in `ExchangeReleaser` carries no reentrancy guard. A nested call supplied with the incremented nonce therefore passes validation and runs to completion inside the outer call.

The nested call is reachable without any privileged role. `release` in `TokenJar` sweeps each asset through `Currency.transfer`, which performs a raw call to the token address, so an attacker who donates an attacker-controlled token to the jar obtains execution during the sweep once a searcher includes that token in the released assets. The asset array is calldata fixed at transaction build time and the jar holds no on-chain asset registry, so the donation must precede the off-chain asset discovery of the searcher. Activating the malicious behavior afterwards, once the searcher has simulated, leaves the releaser nonce untouched, so `InvalidNonce` cannot fire. The reentrant call pays a second threshold and sweeps the jar to an attacker-chosen recipient. Because `release` in `TokenJar` re-reads the live balance of each asset and skips zero balances, the outer sweep then completes as a series of no-ops, and the searcher transaction succeeds having paid the threshold and the fee for assets it never received.

The reentrancy path is acknowledged by the documentation on `release` and by `test_release_reentrantFreshNonceUnprofitable`, but both frame the invariant as the solvency of the jar and treat the resulting loss to the searcher as expected behavior. The impact is bounded, since the attacker also pays a full threshold and cannot control the position of the donated token within the caller-supplied asset array, which determines how much of the jar the reentrant call can capture.

Consider adding a transient storage reentrancy guard to `release` in `ExchangeReleaser`, where no legitimate flow nests, and to `release` in `TokenJar`, which holds the assets and performs the untrusted calls and would therefore remain protected under any future releaser.

Update: *Resolved in pull request #4.*

Notes & Additional Information

N-01 Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- In `ITokenJar.sol`, the `_releaser` parameter of the `setReleaser` function is not documented.
- In `ITokenJar.sol`, the `assets` and `recipient` parameters of the `release` function are not documented.
- In `IResourceManager.sol`, the return value of the `resource` function is not documented.
- In `IResourceManager.sol`, the return value of the `resourceRecipient` function is not documented.
- In `IResourceManager.sol`, the return value of the `threshold` function is not documented.
- In `IResourceManager.sol`, the return value of the `maxReleaseLength` function is not documented.
- In `IResourceManager.sol`, the return value of the `configSetter` function is not documented.
- In `IResourceManager.sol`, the `newConfigSetter` parameter of the `setConfigSetter` function is not documented.
- In `IOptimismMintableERC20.sol`, the return value of the `l1Token` function is not documented.

Consider thoroughly documenting all functions and events that are part of a contract's public API, including all parameters and return values. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Resolved in pull request #7.*

N-02 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `TokenJar.sol`, the `receive` function
- In `IReleaser.sol`, the `IReleaser` interface
- In `IOptimismMintableERC20.sol`, the `IOptimismMintableERC20` interface

Consider thoroughly documenting all functions and their parameters that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should also be clearly documented. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Resolved in pull request #6.*

N-03 Unused Named Return Variables

Named return variables are a way to declare variables that are meant to be used within a function's body for the purpose of being returned as that function's output. They are an alternative to explicit in-line `return` statements.

Within `FeeController.sol`, multiple instances of unused named return variables were identified:

- The `bps` return variable of the `feeConfig` function
- The `recipient` return variable of the `feeConfig` function

Consider either using or removing any unused named return variables.

Update: *Resolved in pull request #5.*

N-04 Resource Recipient May Be Misconfigured As The Releaser

`_setConfig` rejects a zero resource recipient but applies no further restriction, and the `_validateConfig` override in `ExchangeReleaser` rejects only the token jar. A configuration naming the releaser itself as the resource recipient is therefore accepted. `ExchangeReleaser` exposes no mechanism to move tokens out of its own balance and its `_afterRelease` hook is a no-op, so each release would pay the threshold into the releaser

with no means of retrieval. Listing the resource among the released assets does not recover the balance, since `release` in `TokenJar` sweeps the balance held by the jar rather than the balance held by the releaser.

This value is not an arbitrary address. `OptimismBridgedResourceReleaser` requires the resource recipient to equal the releaser address, because its `_afterRelease` override bridges the collected balance to L1. The same value is mandatory in that variant and unrecoverable in the base, and no check distinguishes the two, so a configuration carried between variants reaches this state. The existing rejection of the token jar shows that recipients known to be invalid are already screened at this layer.

Consider rejecting a resource recipient equal to `address(this)` in the `_validateConfig` override of `ExchangeReleaser`, alongside the existing token jar check. The bridged variant replaces that override entirely, so its required configuration remains unaffected.

Update: *Resolved in pull request #8.*

N-05 Typographical Error

In line 25 of `ITokenJar.sol`, the word "callabe" should be "callable".

Consider correcting the identified typographical error to improve the readability of the codebase.

Update: *Resolved in pull request #7.*

Conclusion

TJAR provides a collection and redemption path for protocol revenue, holding accumulated fees in a per-partner jar and releasing them to any party willing to pay a fixed amount of a configured token. OpenZeppelin audited the changes introducing the deployment factory, the chain-wide protocol fee controller, and the OP Stack bridged releaser variant, together with the supporting configuration and nonce logic.

The codebase is compact and its authority surface is narrow. Assets leave a jar only through a single releaser role, and release terms move only through one atomic configuration call that invalidates pending releases. Documentation is extensive, with design rationale recorded alongside the code, and the test suite exercises each in-scope contract, including adversarial token behavior.

Correct operation depends substantially on deployment procedure. Address parity across chains follows from initcode being byte identical, so the fee controller's initial owner and the factory's fee controller argument must be passed identically on every chain, with per-chain governance ownership established afterwards through a transfer that occurs in a single step and cannot be reversed if misdirected. Jar address parity further requires deploying from the same address on each chain, and directing fees to a predicted address before the jar exists assumes the factory has landed at its canonical address there. Beyond deployment, jar ownership remains the system's only trust anchor, since it alone determines which releaser may move assets.

OpenZeppelin thanks the Uniswap team for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.